# Chapter 5 解答

## CH05-Q01

1. Data capacity 是 $2^{16}$ bytes，block size 是 $2^6$ bytes：

$$\#blocks = 2^{16}/2^6 = 2^{10} = 1024.$$

4-way 配置有

$$\#sets = 1024/4 = 256.$$

因此 block offset 是 6 bits，set index 是 8 bits，tag 是 $32 - 6 - 8 = 18$ bits。

2. 對 `0x12345678`：

- offset `0x38 = 56`。
- `(address >> 6) & 0xff = 0x59 = 89`。
- `address >> 14 = 0x48d1`。

對 `0x1a2b3c40`：

- offset `0x00 = 0`。
- set index `(0x1a2b3c40 >> 6) & 0xff = 0xf1 = 241`。
- tag `0x1a2b3c40 >> 14 = 0x68ac`。

3. `0x12345678` 與 `0x12345658` 兩者右移6位的block number相同，block base皆是 `0x12345640`，所以兩者有相同 tag `0x48d1` 與 set index `0x59`，只有 block offset 不同。前者 offset 是 56，後者是 $0x58 \bmod 64 = 24$。若該 block 的 tag 與 valid 仍正確，第二次存取是 hit。

4. Block base 是 `0x12345640`，長度 64 bytes，所以涵蓋 `0x12345640` 到 `0x1234567f`。它放進 set 89，所在 way 由 replacement policy 選。首次 miss 可放入空的 way；若沒有空位，就由 LRU 選擇 victim。

## CH05-Q02

1. Data blocks 固定是 1024。

| Associativity | Sets | Blocks/set | Tag/index/offset | Comparators/set |
|---|---|---|---|---|
| direct | 1024 | 1 | 16/10/6 | 1 |
| 2-way | 512 | 2 | 17/9/6 | 2 |
| 4-way | 256 | 4 | 18/8/6 | 4 |
| fully associative | 1 | 1024 | 26/0/6 | 1024 |

2. 只計 data tag bits：

# Chapter 5 解答 （續）

- Direct：$1024 \times 16 = 16{,}384$ bits。
- 2-way：$512 \times 2 \times 17 = 17{,}408$ bits。
- 4-way：$256 \times 4 \times 18 = 18{,}432$ bits。
- Fully associative：$1 \times 1024 \times 26 = 26{,}624$ bits。

3. Data capacity 不變，但 associative lookup 要比較更多 tags，並用 match/way 選出 data。更多的 comparators、match lines、multiplexer 與 wiring 可能拉長 hit path，也消耗更多 energy。
4. 不一定。Fully associative 通常降低 conflict misses，但可能增加 hit time、tag search energy 與實作難度。AMAT 必須把 hit time 與 miss rate × penalty 一起比較；若 miss-rate 改善不足以抵銷 hit-time 增加，total AMAT 反而更差。

## CH05-Q03

1. Model A 的 AMAT：

$$1 + 0.06(80) = 5.8 \text{ cycles}.$$

2. Model B 的 memory-stall CPI：

$$\Delta\text{CPI} = 0.02(80) + 0.30(0.08)(80) = 1.6 + 1.92 = 3.52.$$

所以

$$\text{CPI} = 1.30 + 3.52 = 4.82.$$

3. 200 million instructions 的 time：

$$T = \frac{2 \times 10^8 \times 4.82}{2.5 \times 10^9} = 0.3856 \text{ s}.$$

4. 這個模型視每次 miss 都造成全額 80-cycle stall。Out-of-order processor 可以在 miss 等待時執行 independent instructions，也可能重疊多個 misses；因此 full-stall 加總通常高估實際 stall。是否可 hiding 取決於 window、dependency、memory-level parallelism 與 contention。

## CH05-Q04

1. 三種 fill time：

- A：$1 + 4(15) + 4(1) = 65$ cycles。
- B：$1 + 2(15) + 2(1) = 33$ cycles。
- C：$1 + 2(15) + 4(1) = 35$ cycles。

2. 每個 block 有 16 data bytes：

# Chapter 5 解答 （續）

- A：$16/65 \approx 0.246$ bytes/bus-cycle。
- B：$16/33 \approx 0.485$ bytes/bus-cycle。
- C：$16/35 \approx 0.457$ bytes/bus-cycle。

3. C 的 wide memory 可把兩次 access startup 合併，但 bus 仍逐 word transfer。它降低 access latency，保留 transfer bottleneck。若 data volume 與 burst sequentiality 足夠，C 比 A 好；若 bus bandwidth 才是主限制，widening bus 的收益較大。
4. Startup 增加到 40 cycles 後：

- A：$1 + 4(40) + 4 = 165$。
- B：$1 + 2(40) + 2 = 83$。
- C：$1 + 2(40) + 4 = 85$。

B 與 C 幾乎相同，兩者相對 A 的倍率都約為 2；降低 startup 次數的兩配置都獲得主要好處。不能只把 A 的倍率寫成 startup ratio 而忽略 transfer。

## CH05-Q05

1. Global L2 miss rate 是所有 references 中最後仍 miss 到 memory 的比例：

$$M_{2,\text{global}} = M_1 M_2 = 0.04(0.25) = 0.01.$$

2. 完整 AMAT：

$$\begin{aligned}\text{AMAT} &= T_1 + M_1 T_2 + M_1 M_2 P_{\text{mem}} \\ &= 1 + 0.04(8) + 0.04(0.25)(100) \\ &= 1 + 0.32 + 1.00 \\ &= 2.32 \text{ cycles}.\end{aligned}$$

3. 若把 25% 錯當 global L2 miss：

$$1 + 0.04(8) + 0.25(100) = 26.32 \text{ cycles}.$$

這是嚴重高估，因為 25% 是「L1 miss 中又 miss L2」的 local rate。

4. L1 misses 中，$1 - 0.25 = 75\%$ 由 L2 hit 服務。若 $M_1 = 0.02$、local $M_2 = 0.25$，global L2 miss 變成

$$0.02(0.25) = 0.005.$$

## CH05-Q06

1. 4 KiB $= 2^{12}$，所以 page offset 是 12 bits：

$$\text{VPN} = 0x00403, \qquad \text{offset} = 0xabc = 2748.$$

# Chapter 5 解答 （續）

2. 32-bit VA 有 $32 - 12 = 20$ 個 VPN bits，因此單層 page table 有 $2^{20}$ entries。每 entry 4 bytes：

$$2^{20} \times 4 = 2^{22} = 4\ \mathrm{MiB}.$$

3. 兩層各 10 bits：

- level-1 index：`(VA >> 22) & 0x3ff = 0x001`。
- level-2 index：`(VA >> 12) & 0x3ff = 0x003`。

4. PA 由 PPN 與原 offset 拼接：

$$\mathrm{PA} = (0x0005f \ll 12) \mid 0xabc = 0x5f000 + 0xabc = 0x5fabc.$$

PTE 的 write bit 為 0，store 是 protection fault，不是 page-not-present fault。OS 可以終止 process、送 signal 或依合法機制改變權限，但不能在權限未允許前完成 store。

## CH05-Q07

1. Frame F 的 old page W dirty，所以必須先把 W 的完整內容寫回 backing storage，或在可安全完成的前提下記錄 pending writeback。只有在確認 F 的 content 已保存且 frame ownership/PTE 狀態更新後，才能覆蓋 F 並載入 V。
2. 4-byte store 在 page fault 解決前可能被中途中斷。若 cache 或 memory 先留下 partial architectural write，而 fault 最終不可恢復，程式會看到不存在的 write；若 fault 可恢復，也必須避免 store 被執行兩次或只執行一半。Hardware/OS 必須保存 faulting VA、access type、data 與 precise state，完成 mapping 後再重新執行 store。
3. 若 TLB 內有舊的 V mapping、shared/invalid translation 或因 frame remap 而 stale 的 PPN，TLB hit 可能使用舊 physical address。OS 要 invalidate matching entry，或使用 generation/ASID 規則確保新 mapping 不被舊 entry 遮住。
4. ASID 把 process identity 加入 TLB tag。不同 processes 的相同 VPN 可以同時存在，context switch 後不必全部 flush。ASID 只區分 translation identity，不取代 PTE/TLB 的 read/write/user 權限檢查，也不取代 cache coherence。

## CH05-Q08

1. Clean read miss，Allocate總共三個cycles，前兩個not-ready、第三個Ready：

# Chapter 5 解答 （續）

```
cycle 1: Idle, request read -> Compare Tag
cycle 2: miss, victim clean -> Allocate
cycle 3: Ready=0 -> Allocate
cycle 4: Ready=0 -> Allocate
cycle 5: Ready=1, block installed -> Compare Tag
cycle 6: valid and tag hit, read selected word
         set Cache Ready -> Idle
```

本題已明定每個Allocate cycle的Ready時點；若改成三個not-ready cycles加一個Ready cycle，總Allocate時間才會變四cycles。

2. Dirty write miss，write-back 與 allocate 各一個 Ready cycle：

```
Idle, request store
-> Compare Tag, miss and old victim dirty
-> Write-Back, Ready=1
-> Allocate, Ready=1
-> Compare Tag, valid/tag hit
    write store data, set dirty, set Cache Ready
-> Idle
```

3. Controller 在 `Idle` 接受 request、轉到 `Compare Tag` 的時脈邊緣即保存 address、request type 與 store data，並在整筆 transaction 完成前保持；不能等 miss 後才依賴可能已改變的 processor inputs。若未保存，`Allocate` 完成後無法知道要回傳 read 或寫入 store data；可用 request register、transaction queue 或 dedicated state。
4. Allocate 會覆蓋 victim frame 的 data/tag/dirty。若 dirty victim 尚未保存或放入有可靠性保證的 write buffer，最新修改會永久遺失，main memory 仍可能是舊值。被破壞的是 architectural memory state 與 recovery invariant，不只是效能。

## CH05-Q09

1. Write-back MSI 狀態：

| Step | Event | P1 X | P2 X | Memory X |
|---|---|---|---|---|
| 0 | initial | I | I | old |
| 1 | P1 read miss | S(old) | I | old |
| 2 | P2 read miss | S(old) | S(old) | old |
| 3 | P1 write X=1 | M(1) | I | temporary stale |
| 4 | P2 實際 read X；read miss / BusRd | S(1) | S(1) | 1 |

# Chapter 5 解答 （續）

2. Step 3 是 write miss / upgrade。P1 必須先取得 exclusive permission，將 P2 的 valid `S(old)` copy invalidate；只有在 P2 的 copy 不再可讀後，P1 才能持有唯一 valid copy 並進入 `M(1)`。
3. Step 4 的 supplier 是原本持有 `M(1)` 的 P1，不是 stale main memory。P1 提供 1，P1 由 `M(1)` 降為 `S(1)`；requester P2 由 `I` 進入 `S(1)`。此 write-back MSI 的這個 read 同時更新 memory 為 1。若 protocol 另有 owner/owned state，才可能讓 memory 暫時 stale；那不是本題的三段式 MSI。
4. `M` 的定義是該 block 只有此 cache 有 valid copy；若同時讓另一 cache 保持 valid `S`，則在 owner 寫入或尚未 write-back 時，至少兩份 copy 可以不同步。`S` 的定義因此要求 memory 為最新。Coherence 對同一位置提供 read value、write serialization 與 invalidation；memory consistency model 再規定跨位置及 synchronization 的 ordering。沒有 synchronization 的 racing access 仍可能依 memory model 看到不同次序。

## CH05-Q10

1. Block size 是 16 bytes，2-block direct-mapped 的 index 是 block number mod 2。

- `0x1000`：block `0x100`，index 0，first access 是 cold miss。
- `0x1020`：block `0x102`，index 0，first touch 是 cold miss，同時因 set competition 淘汰第一個 block。
- 第二次 `0x1000`：index 0 的 block 已不在，miss 是 conflict miss。

2. Fully associative 2-block cache：

- `0x1000`：cold miss，放入。
- `0x1040`：cold miss，放入。
- `0x1080`：cold miss，cache full，必須替換其中一個。
- 第二次 `0x1000`：本題指定LRU，第3次access淘汰最久未用的`0x1000`，所以第4次必然miss，歸類為capacity；若未指定replacement，其他策略可能保留它，不能僅由容量判定這一筆必然miss。

3. 改成 2-way、相同 capacity 只有一個 set，`0x1000` 與 `0x1020` 可同時存在。Sequence 得到 first A cold miss、B cold miss、second A hit；不再有 second A 的 conflict miss。
4. 提升 associativity 或加入 victim/lower-level cache 可降低 conflict。代價包括更多 tag comparators/mux、較長 hit path、更高 energy 與面積。是否值得要以 AMAT 與 workload 實測，而非只比較 hit count。

# Chapter 5 解答 （續）

## CH05-Q11

1. 狀態如下。* 表示 dirty；MRU/LRU 在每步更新。

| Step | Access | Way 0 | Way 1 | MRU / LRU | Lower action |
|---|---|---|---|---|---|
| 0 | initial | empty | empty | — | — |
| 1 | R A | A | empty | A / — | read A |
| 2 | R B | A | B | B / A | read B |
| 3 | W A | A* | B | A / B | none |
| 4 | W C | A* | C* | C / A | evict clean B；read C |
| 5 | W D | D* | C* | D / C | evict dirty A；write A then read D |

2. Reads 是 A、B、C、D 共 4 blocks；dirty write-back 是 A 共 1 block；total 5個64-byte block transfers，共320 bytes。Way0由A直接替換成D，Way1的C不搬動。W(A) 是 write hit，在 write-back policy 下不立即寫下層。
3. Write-through、no-write-allocate 路徑：

- R(A)、R(B) 各讀一個 block。
- W(A) 仍為 hit，立即 write-through A。
- W(C)、W(D) 都是 no-allocate misses，直接 write-through C、D，不改變 set。
- Set 最後保留 A、B。Lower traffic是2個64-byte reads加3個4-byte writes，共 128+12=140 bytes。5次動作不是5個等長blocks；若要求WB末尾C、D寫回，還需另加 128 bytes，本題沒有這項終態要求。

4. Hit detection 只由 valid、tag 與 index 決定；dirty 只決定 valid copy 是否已同步到下層。若 evict dirty A 前覆寫其 storage，A 最新資料沒有其他副本，main memory 仍是舊值，architectural memory 就遺失已提交的 store。

## CH05-Q12

1. Page offset 是 12 bits：

$$\mathrm{VPN} = 0x0badf, \quad \mathrm{offset} = 0x00d.$$

32 KiB data cache 有 $32768/32 = 1024$ blocks；2-way 有 512 sets。Offset 5 bits、index 9 bits，因此PIPT tag是 $32 - 5 - 9 = 18$ bits。Index 可寫成 `(VA>>5)&0x1ff`，offset 是 `VA&0x1f`。

# Chapter 5 解答 （續）

2. Index佔bits13:5，5+9=14>12，不完全在page offset內。VA `0`與`0x1000`分別選set0、128，即使它們映到同一PA也落在不同sets。若僅比PA[31:14]會漏掉physical bits13:12，可誤認不同PA為hit；須比完整20-bit PPN或有等價機制，但完整tag仍不能消除跨set別名。32 KiB/ways必須不超過4 KiB，至少8-way；8-way有128sets，5+7=12。不能把未給定的page-coloring假設加進原題。
3. VPN 低 10 bits 是 level-2 index：

$$\mathrm{L1index} = (0x0badf \gg 10) \& 0x3ff = 0x2e,$$
$$\mathrm{L2index} = 0x0badf \& 0x3ff = 0x2df.$$

L1 PTE address：

$$0x00800000 + 0x2e \times 4 = 0x008000b8.$$

L1 PTE 的 PPN `0x000031` 形成 L2 table base `0x00031000`；L2 PTE address：

$$0x00031000 + 0x2df \times 4 = 0x00031b7c.$$

L2 PPN `0x0001a` 與 page offset `0x00d` 拼成

$$PA = (0x0001a \ll 12) | 0x00d = 0x1a00d.$$

Read 可依權限完成；store 因 `write=0` 產生 protection fault，不能先寫 cache 或 memory。 4. TLB miss total：

$$2 \times 25 + 10 + 3 = 63 \text{ cycles}.$$

Miss/hit access-time ratio：

$$63/3 = 21.$$

此 63 cycles 只含題目明定的 page-table reads、固定管理成本與 data-cache lookup；真正 page fault、disk I/O、block fill 與 contention 另計。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Chapter 5 解答 （續）

## 原創驗算程式附錄: cache_address_lab.py

```
"""Verifiable teaching examples for cache decoding and page translation."""

from dataclasses import dataclass


@dataclass(frozen=True)
class AddressFields:
    tag: int
    set_index: int
    offset: int


@dataclass(frozen=True)
class CacheSpec:
    capacity_bytes: int = 64 * 1024
    block_bytes: int = 64
    ways: int = 4
    address_bits: int = 32

    @property
    def blocks(self) -> int:
        return self.capacity_bytes // self.block_bytes

    @property
    def sets(self) -> int:
        return self.blocks // self.ways

    @property
    def offset_bits(self) -> int:
        return self.block_bytes.bit_length() - 1

    @property
    def index_bits(self) -> int:
        return self.sets.bit_length() - 1

    @property
    def tag_bits(self) -> int:
        return self.address_bits - self.offset_bits - self.index_bits

    def decode(self, address: int) -> AddressFields:
        offset_mask = self.block_bytes - 1
        index_mask = self.sets - 1
        return AddressFields(
            tag=address >> (self.offset_bits + self.index_bits),
            set_index=(address >> self.offset_bits) & index_mask,
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Chapter 5 解答 （續）

```
            offset=address & offset_mask,
        )


class SetAssociativeCache:
    def __init__(self, spec: CacheSpec) -> None:
        self.spec = spec
        self.entries: list[list[tuple[int, int]]] = [[] for _ in range(spec.sets)]
        self.clock = 0

    def access(self, address: int) -> bool:
        self.clock += 1
        fields = self.spec.decode(address)
        ways = self.entries[fields.set_index]
        for position, (tag, _) in enumerate(ways):
            if tag == fields.tag:
                ways[position] = (tag, self.clock)
                return True
        if len(ways) < self.spec.ways:
            ways.append((fields.tag, self.clock))
        else:
            victim = min(range(len(ways)), key=lambda i: ways[i][1])
            ways[victim] = (fields.tag, self.clock)
        return False


@dataclass
class CacheLine:
    tag: str
    last_used: int
    dirty: bool = False


def write_back_trace(operations: list[tuple[str, str]], ways: int = 2) -> dict:
    lines: list[CacheLine] = []
    read_blocks: list[str] = []
    write_backs: list[str] = []
    snapshots: list[dict] = []
    clock = 0
    for operation, tag in operations:
        clock += 1
        matched = next((line for line in lines if line.tag == tag), None)
        if matched is not None:
            matched.last_used = clock
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Chapter 5 解答 （續）

```
            if operation == "W":
                matched.dirty = True
            snapshots.append({"event": f"{operation} {tag}", "result": "hit", "lines":
[line.tag for line in lines]})
            continue

        if len(lines) == ways:
            victim = min(lines, key=lambda line: line.last_used)
            if victim.dirty:
                write_backs.append(victim.tag)
            lines.remove(victim)
        lines.append(CacheLine(tag=tag, last_used=clock, dirty=operation == "W"))
        read_blocks.append(tag)
        snapshots.append({"event": f"{operation} {tag}", "result": "miss", "lines": [line.tag
for line in lines]})
    return {
        "snapshots": snapshots,
        "read_blocks": read_blocks,
        "write_backs": write_backs,
        "set": [{"tag": line.tag, "dirty": line.dirty} for line in sorted(lines, key=lambda
line: line.last_used)],
    }


def translate_two_level(va: int, ppn: int | None, page_bits: int = 12) -> tuple[int, int,
int, int, int | None]:
    vpn = va >> page_bits
    offset = va & ((1 << page_bits) - 1)
    level1 = (vpn >> 10) & 0x3FF
    level2 = vpn & 0x3FF
    pa = None if ppn is None else (ppn << page_bits) | offset
    return vpn, offset, level1, level2, pa


def main() -> None:
    spec = CacheSpec()
    assert (spec.blocks, spec.sets) == (1024, 256)
    assert (spec.tag_bits, spec.index_bits, spec.offset_bits) == (18, 8, 6)

    first = spec.decode(0x12345678)
    second = spec.decode(0x1A2B3C40)
    same_block = spec.decode(0x12345658)
    assert first == AddressFields(0x48D1, 0x59, 0x38)
    assert second == AddressFields(0x68AC, 0xF1, 0)
    assert (same_block.tag, same_block.set_index, same_block.offset) == (0x48D1, 0x59, 0x18)

    cache = SetAssociativeCache(spec)
    addresses = [0x12345678, 0x1A2B3C40, 0x2BADF00C, 0x40001020, 0x12345658]
    results = [cache.access(address) for address in addresses]
```

# Chapter 5 解答 （續）

```
    assert results == [False, False, False, False, True]

    vpn, offset, level1, level2, pa = translate_two_level(0x00403ABC, 0x0005F)
    assert (vpn, offset, level1, level2, pa) == (0x403, 0xABC, 1, 3, 0x5FABC)
    assert translate_two_level(0x00403ABC, None)[-1] is None

    write_trace = write_back_trace([("R", "A"), ("R", "B"), ("W", "A"), ("W", "C"), ("W",
"D")])
    assert write_trace["read_blocks"] == ["A", "B", "C", "D"]
    assert write_trace["write_backs"] == ["A"]

    vipa = 0x0BADF00D
    ptw_vpn = vipa >> 12
    l1_index = (ptw_vpn >> 10) & 0x3FF
    l2_index = ptw_vpn & 0x3FF
    l1_pte_address = 0x00800000 + l1_index * 4
    l2_pte_address = (0x000031 << 12) + l2_index * 4
    final_pa = (0x00001A << 12) | (vipa & 0xFFF)
    assert (ptw_vpn, l1_index, l2_index) == (0x0BADF, 0x2E, 0x2DF)
    assert l1_pte_address == 0x008000B8
    assert l2_pte_address == 0x00031B7C
    assert final_pa == 0x1A00D
    assert 2 * 25 + 10 + 3 == 63

    amat = 1 + 0.04 * 8 + 0.04 * 0.25 * 100
    assert abs(amat - 2.32) < 1e-12

    print("cache: 64 KiB, 4-way, 64-byte blocks")
    print(f"blocks={spec.blocks}, sets={spec.sets}, tag/index/offset=
{spec.tag_bits}/{spec.index_bits}/{spec.offset_bits}")
    for address, result in zip(addresses, results):
        fields = spec.decode(address)
        status = "HIT" if result else "MISS"
        print(f"0x{address:08x}: tag=0x{fields.tag:x}, index=0x{fields.set_index:x}, offset=
{fields.offset:2d}: {status}")
    print(f"translation: VPN=0x{vpn:05x}, L1={level1}, L2={level2}, PA=0x{pa:05x}")
    print(f"AMAT model: {amat:.2f} cycles")
    print(f"write-back trace: reads={write_trace['read_blocks']}, write-backs=
{write_trace['write_backs']}")
    print(f"page walk: L1 PTE=0x{l1_pte_address:08x}, L2 PTE=0x{l2_pte_address:08x},
PA=0x{final_pa:x}")
    print("cache address lab: PASS")


if __name__ == "__main__":
    main()
```

# Chapter 5 解答 （續）

**參考資料與署名**

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study